TECHNISCHE HOCHSCHULE MITTELHESSEN

CAMPUS GIESSEN

MNI
Mathematik, Naturwissenschaften und Informatik

Master Thesis

Reengineering and analysis of a model interpreter implementation in the domain of mobile applications

zur Erlangung des akademischen Grades

Master of Science

eingereicht im Fachbereich Mathematik, Naturwissenschaften und Informatik an der Technische Hochschule Mittelhessen

von

Samuel Schepp

March 26, 2019

Referent: Prof. Dr. Peter Kneisel

Korreferent: Dennis Priefer

Erklärung der Selbstständigkeit

Hiermit versichere ich, die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie die Zitate deutlich kenntlich gemacht zu haben.

Gießen, den March 26, 2019 Samuel Schepp

This thesis describes configuration-based mobile applications and their infrastructure. In addition, a possible way of implementing a configuration system is discussed. Finally, the implementation is compared to the approach of established literature by classifying the model and interpreter.

Furthermore, there is an evaluation of the use case of interpreting and generated apps.

Contents

1 Introduction 1
- 1.1 Problem definition . . . 1
- 1.2 Motivation . . . 2
- 1.3 Limitation . . . 3
- 1.4 Structure . . . 3
  - 1.4.1 Logical Approach . . . 3
  - 1.4.2 Goals . . . 4

2 Background 5
- 2.1 Company . . . 5
  - 2.1.1 Markets . . . 5
  - 2.1.2 City Marketing . . . 6
- 2.2 Model Interpreter System . . . 6
  - 2.2.1 Use Cases . . . 7
  - 2.2.2 Interoperability . . . 7
- 2.3 Models . . . 8
  - 2.3.1 General Model Theory . . . 8
  - 2.3.2 Specialized Model Theory . . . 9
- 2.4 Model Driven Development . . . 9
  - 2.4.1 Modeling a Software System . . . 10
- 2.5 Code Generators . . . 12
  - 2.5.1 Meta Programming . . . 12
  - 2.5.2 Generation . . . 13
- 2.6 Model Interpreters . . . 19

3 Concept 21
- 3.1 Implementation Goals . . . 21
  - 3.1.1 Requirements . . . 21
  - 3.1.2 Drawbacks . . . 21
  - 3.1.3 Iterative Development . . . 21

4 Implementation 23
- 4.1 Language . . . 23

4.2 Components . . . 23
4.2.1 Sub-Models . . . 23
4.2.2 Layout . . . 23
4.2.3 Persistence . . . 23
4.2.4 Interaction . . . 23
4.3 Infrastructure . . . 23
4.3.1 Control Center . . . 23
4.3.2 Data Backend . . . 23
4.3.3 Resource Server . . . 23
4.3.4 Image Cruncher . . . 23

5 Evaluation 25
5.1 Requirements . . . 25
5.2 Analysis . . . 25
5.2.1 Classification . . . 25
5.2.2 Approach . . . 25
5.2.3 Model . . . 25

6 Conclusion 27
6.1 Summary . . . 27
6.2 Discussion . . . 27
6.3 Next Steps . . . 27
6.3.1 Hybrid Approaches . . . 27
6.3.2 Runtime Models . . . 27
6.3.3 Abstraction . . . 27

Bibliography 29

Index of abbreviations 31

List of Figures 33

List of Tables 35

Listings 37

1 Introduction

Mobile application and digital media usage is still on the rise[Per]. ”Apps” are one of many possible ways to interact with customers and consumers. A web appearance on social media or a dedicated web page can also fulfill these needs.

A mobile application can enable user specific features without logging in, because the user can be identified with an identification token, that is generated, while the user installs the app. It is possible to activate special features like geo-fencing, push notifications or local data storage, that is synchronized with the platform specific cloud services (e.g. Apple iCloud or Google Drive), without any custom user authentication.

A mobile app is integrated more deeply into the device, than a web app. However, web browsers already support a lot of the features by now, that originally were only available to native apps[1]. The current development indicates, that the differences will be further eliminated by the browser vendors.

1.1 Problem definition

Mobile applications have significant disadvantages, that push the development of web apps forward. Changes have to be approved by the phone vendors, in order to be able to release updates to the app. This causes development delays and reduces the ability to react to events or customer requirement changes.

All apps have to be kept in good condition, which creates more work, the more apps there need to be maintained. New operating system features have to be implemented in all of the apps, which can cause inconsistencies. All apps have to be tested in detail, to make sure that all apps implement the required change correctly. Platform changes can only be dealt with one by one. Developers need deep system knowledge to develop the apps. Every change has to go through one of each platform experts, so development is expensive.

1 E. g. Local storage, push notifications

An example for a change, that all apps should implement is the display notch. It causes the screen to extend to the edge of the display. However some of the screen is cut out to make room for sensors and the camera.

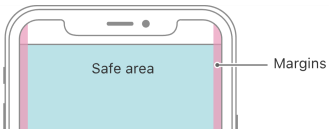


Figure 1.1: Display notch with safe area, Apple iPhone X

The notch is integrated to the default interface framework, so that developers can react accordingly and use the provided margins. This change requires all apps to implement a correct way to handle that. Developers have to go though every screen and correct the view.

1.2 Motivation

To be able to deliver the full mobile application experience, the app has to be native. This will change, when all of the phone's hardware interface is exposed to the web browser and user interface performance is better. But for now, the only way to use these features and be good on battery life and data usage is by providing a native app, that is written in the native language and uses features, directly provided by the phone vendors.

Web app wrapper like "ionic" are able to use these features and provide their own interface to the web app. In this way, a web app can in fact use restricted components like push notifications and beacons.

With this approach, every new app still has to be developed and old ones have to be maintained individually. In addition, it involves the dependence on third party framework vendors. Building apps with ionic creates a massive list of dependencies: ionic, angular, npm and nodejs. Native apps have the ability to reduce reliance, by only using native libraries, which are very comprehensive by default.

To be able to create many apps with little effort, a model driven approach can be considered. A well abstracted model can have the ability to be easy to understand by users and the platforms can bring their respected implementation in, so any app looks and feels native. A service can deliver pre-rendered sub-models, so that complex client workload can be moved to the server.

Existing apps are easy to maintain, because there is only one code base per platform. One bug fix applies to all apps. Also, it is easy for anyone to change small things like a

label, a background color or the sorting of a list. The goal is, to be able to configure new apps in one day, together with the customer.

1.3 Limitation

This thesis will not describe the development experience nor the initial expense of the project. Neither will this thesis evaluate financial aspects and at which point, the project is worth the development costs. The domain examination will be narrowed down to the two main ones, mobile phones and digital signage.

1.4 Structure

The thesis will give some information on the background of the interpreter. The company Fabrik19 GmbH, which is actively developing and maintaining an implementation, will be described and characterized. After that, the general concept of model driven development, code generators and interpreters are examined and presented.

The main content will then be about the concrete implementation, by describing the most significant design choices, overall infrastructure and usage examples. The focus will be on the two main platforms 'iOS' and 'Android'. The web and *.NET* clients are not considered, because they are not targeted for mobile devices and only support a subset of the model.

In the final chapter, the concept is applied to the implementation, to see how it fits the theoretical model of interpreters. Differences are worked out and discussed. Also, an outlook will be given, where additional topics, like hybrid approaches, runtime models and abstraction are covered.

1.4.1 Logical Approach

To be able to evaluate the interpreter, requirements are listed and described. These will be used to validate the implementation.

Since the implementation for the iOS platform has been re-engineered to meet current demands, the problems with the old version will be outlined and compared to the new one.

1.4.2 Goals

This thesis will show how a model interpreter can be evaluated against requirements and informal criteria. In addition, the re-engineering and model development processes will be compared to common approaches in modern literature.

The following questions will be answered by applying theoretical approach to the given example.

> *Which requirements are placed on the model interpreter implementation?*
>
> *How can the specific model interpreter be implemented?*
>
> *How can the implementation be evaluated on the basis of the requirements?*

2 Background

The system, that is discussed here, is a model interpreter that is designed to run mobile applications on iOS and Android. The system, which includes the interpreters and the backends are called mos[1]. It is in active development and powers over 50 apps that all are available in the Apple AppStore and Google PlayStore. The app categories range from retail to event to city marketing apps[2].

2.1 Company

The creator of the system is Fabrik 19 GmbH. The company is based in Gießen, Hesse, Germany and was founded 2013 by Mark Pralle and Sven Herchenhein. At the moment, the company has bout 25 employees. Besides of apps, there are also other projects and solutions, like consulting and web hosting.

2.1.1 Markets

Fabrik19 does not deliver apps directly to customers, but is partner of retail corporations and city marketing agencies. The current flagship is the Gießen App, which was developed in cooperation with DISTAMA GmbH. The app was downloaded 34,000 times and has 20 distinct and active users per day.

Besides mobile apps, digital signage is another important market. The companion to the Gießen App is a stele that contains a large touchscreen and runs an app that is optimized for big touchscreens. It is located on the main pedestrian shopping street in Gießen and provides respective information on the stores and points of interests[3]. Other stele systems are also active in Globus hardware stores and Globus grocery stores.

1 Mobility Suite
2 `https://fabrik19.de/technologie/`
3 `https://distama.de/bid-seltersweg/`

2.1.2 City Marketing

Since the apps use live data and have the ability to change it's content structure, an active support and maintenance is needed to keep the app alive.

To be able to advertise and publish city marketing apps as a service, Distama GmbH was founded which is a subsidiary firm of Fabrik19. The goal was to combine technology knowledge with marketing, to be able to provide the app and the attached customer support. City marketing agencies are not always able to support an app, because they lack experience and domain knowledge. Distama is the mediator between the developer, the customer and the user.

2.2 Model Interpreter System

The main development focus lays on the app interpreter platform "mos". It supports the two major platforms iOS and Android, but there is also a lightweight version available in .Net, which is running on the steles as a Windows app. A web client is also actively in development, which aims to replace the .Net version.

A configuration are multiple JSON files, that represent the app meta information, its layout and its behavior. The files are called node. There is one root node, whose url is baked into the app while compiling. This root node defines the first screen of the app. All the other screens can be defined in other nodes. When displaying another node, the new node is loaded from cache or network and its new view is pushed on the view stack, which is done by the interpreter. On the particular platforms, this action will look different and will use the platforms native animations. Web apps like Ionic have their own push animations which makes them feel foreign. The previous node can pass data, like the ID of the item that should be displayed.

The goal is, to be able to create a new app in one day, by meeting with the customer and configuring the app together. The syntax of the configuration is designed to abstract all of an apps standard behavior. Because of a custom layout system, that can be described with constraints and will layout automatically, it is not possible to recognize that it is only a configured app, instead of a custom native one.

The apps have sophisticated caching and optimization mechanisms, which makes it possible to use them completely offline, without any additional configuration effort. The configuration, images and data are cached heavily, which makes the apps very fast. But because of the dynamic design of the configuration syntax, it is possible to clear the caches and force new versions. The cache first policy of the interpreter makes sure, that contents are always available and loads changes in the background.

Besides of displaying data, the client is highly intractable and can perform various custom tasks on the device, like uploading local data, saving data to the devices user settings and providing push, beacon and location infrastructure.

This data can be used in describing the layout by defining layer conditions, which makes the app behave accordingly.

2.2.1 Use Cases

The main use cases are marketing and customer interaction. The concept is data source driven and enables configuration of any data source. Because of a template engine, data can be arranged for display in an intuitive way.

Since the configuration can be changed on the fly and without the need of providing an update via app store, the contents can be adjusted to fit current events like holidays or promotions. Especially for digital signage it is important to be able to change the contents without any application updates.

Any change to the configuration is reflected as soon as the app checks for configuration changes. The update mechanism are also highly customizable, so that the optimal update intervals and strategies can be used for different apps.

A city marketing app might need a medium update interval, so that promotions and special content are available when needed. A event app should have a very short update interval, so that the events program changes or announcements are delivered as fast as possible.

2.2.2 Interoperability

The app can be filled with data via common REST services, that deliver JSON data. The app uses HTTP mechanisms and a custom delta system to only load new data. The mos backend system also has a data proxy, which can load any data and provide it in a simpler, preprocessed format, so it is simple for the apps to download. This way, it is possible to use any data source and still be able to change the templates and layouts without updates.

The apps have the capability to be integrated in a Wordpress instance, a chat service like Mattermost or an secured API[1] that uses authentication methods like client certificates, authorization headers or JSON web tokens.

1 Application Programming Interface

By default any network communication is secured with an app specific client certificate. Other authentication methods have to be configured.

2.3 Models

Since the mos-Framework tries to abstract mobile application platforms into a common language, the configuration could be considered to be an abstract model of a mobile application. To classify and evaluate the model approach for this domain, the following section will discuss models and their properties.

2.3.1 General Model Theory

A general model is described by Herbert Stachowiak[1]. He defines three properties, that apply to all models.

Abbildungsmerkmal (Representation): A model is a representation of an original, that is either a natural or artificial thing, that can have properties and behavior. The original can be a model it's self.

Verkürzungsmerkmal (Reduction): The second property is the ability to reduce properties and behavior of the original to a set, that is relevant for the model handlers. The original might be a complex system and only a subsystem or specific properties and their interaction are relevant. For this, the model can reduce the complexity, to be more simple to understand and to minimize the error rate.

Pragmatisches Merkmal (Pragmatism): Models can not be clearly assigned to their originals. That means, that original and model are not equal, but can be interchanged. There are three cases defined by Stachowiak, in which the model can be used as an replacement. Models are not only models of something, but they are also models for someone, a person or an artificial model user. They perform their functions within a time interval. And they are models for a specific purpose. Often times, the purpose is an action that is not possible on the original because it may be to complex, or an physical original could be destroyed.

If theses properties apply an object, i can be called model according to Stachowiak. It can be applied by numerous of common usages of the word. Mathematical models, philosophical models, simulations and concrete object models are some examples, that fit the requirements.

1 [Staa] 131

2.3.2 Specialized Model Theory

To classify models in a more specific way, they can be categorized as descriptive and prescriptive models. A descriptive model represents an existing system, as it describes it's current behavior. The general model specifications can still be applied, since all three properties are true for such a model. It cannot be modified unless the original changes, since it is representing it directly. A change to one of them without updating the other would result in the model not representing the original, which makes it a prescriptive model. A change on the original which is not relevant to the model will keep its validity.

Models, that represent a fictive object or system are called prescriptive models. The prefix "pre-" indicates, that the model exists before the original. The model is then used to prototype a system, and test their behavior and properties, without the need to create the full system.

According to Seidewitz 2003 [Sei] both descriptive and prescriptive models define a set of originals. That means, that a original can be derived from a model by adding additional information that does not contradict each other. Often times, the information that must be added is trivial or can be fetched from somewhere else. That is possible, because the model has by definition all of the necessary information.

A derived original fulfills all of the models properties, otherwise the original is not a correct derivation.

2.4 Model Driven Development

Models can be used in a software development process (MDSD[1]) or as part of a software architecture (MDA[2]). Software is also a model, which represents real life objects and procedures. Since it is well structured by default, it is fairly easy to create model from a existing code basis. UML[3] is a widely used modeling languages, which can represent static, dynamic and interactive aspects of software systems.

The problem is, that the domain specific structure of the code can be hard to understand, because the abstrcation level of a modern object oriented programming language is much lower, than the domain requires. VÖLTER and STAHL describe the result of scattering relevant information in the code as: 'The much-treasured inner structure is present in a

1 Model Driven Software Development
2 Model Driven Architecture
3 Unified Modeling Language

cloudy, distributed, and of course also a strongly individualized form.'[1] That means, that the domain's structure, components and behavior definition is represented in the software code, but in a way that is hard to understand. That is, because the relevant parts and the domain critical definitions are spread in the software's code. It is also harder to unterstand a domains properties, by only reading the programming language.

A software model, like an UML class diagram are an old tool to document and model software systems. The problem here is, that the abstraction level of an UML class diagram is often times the same as the programming language itself. A developer could find himself transforming a class into a diagram or generating a diagram, which does not contribute to the understanding of the domain structure. This happens, because the programming language itself is abstracted, but not the domain itself.

To address the problem of widely spread domain implementations, the programs important and relevant structure is compressed in a model, which defines and represents the program on a abstract and formal level. The model can be a textual DSL[2], a graphical diagram or even an informal text, that is then converted into a concrete syntax. It contains all the domain relevant information, which is required to derive a full implementation from it.

This procedure results in a model becoming part of the source of a program, and not only the documentation. A good model syntax can lead to the model being the documentation itself.

2.4.1 Modeling a Software System

An existing implementation can be analyzed, to create a model from it. If no model infrastructure exists, this is the way to approach the transformation.

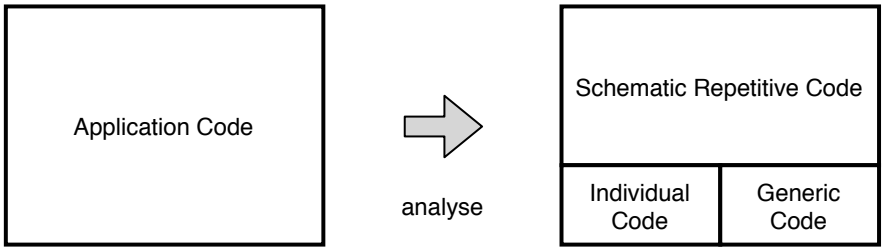


Figure 2.1: Analyzing an existing implementation

Figure 2.1 shows, how the code basis of an existing implementation of a software system can be split into three categories[3].

1 [Stab] 14
2 Domain Specific Language
3 [Stab] 15

The **schematic repetitive code** is the part, that needs to be abstracted into a model. It contains the application specific logic, that is later modeled by a user. It does not contain code, that is equal for all applications in the same domain.

Generic code is derived from the domain and target platform and not from the concrete application. Therefore, it is not modeled, but generated by the domain's MDD[1] infrastructure. It is mostly boilerplate code, that should not be part of the model. It is the same for every application and is later part of the generator templates.

Individual code is unable to be represented in the model, because it is to specific and does not fit into the domain. The tools need to make sure that this code can be integrated into the generated code later. To simplify the MDSD process, individual code should be avoided as far as possible, because in the worst case the individual code makes up all of the application code, which eliminates the possibility to use a model to represent the application. In the best case, the individual code portion is empty. This allows for a more simpler tools, because the application can be generated completely out of the model and there is no need for individual code interfaces.

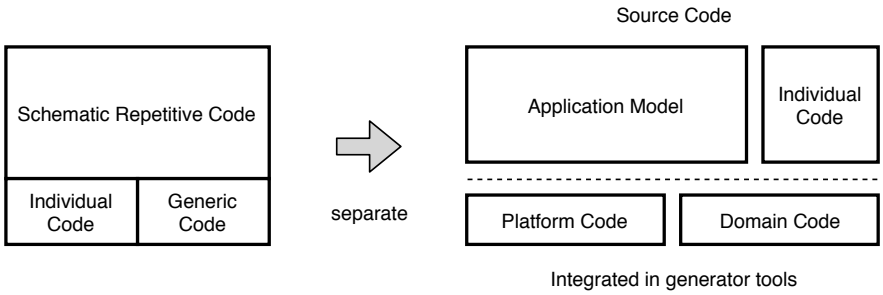


Figure 2.2: Separation of code into source code and steady code

Figure 2.2 shows, how the analyzed parts can now be separated into source code and steady code. Source code describes the part of the code that is application specific and is developed by the MDD user. The application model and individual code should be checked into source control, since that is the actual source of the application, given a MDD infrastructure exists.

Platform and domain code is part of the MDSD tools. It is baked in, so that it is part of the output while generating. In case of code generators, this code would be part of the templates. In theory, it is now possible to develop any app, that fits into the domain, by only defining the model and the individual code. Extensions of the model's capabilities might need to be reflected in the platform or domain code.

1 Model Driven Development

2.5 Code Generators

Now that there is a model of an application, a concrete implementation has to be created from the model. Code generators can take the model and generate a software system, that is an instance if the model. Another approach are model interpreters, which execute the model directly, similar to a virtual machine.

2.5.1 Meta Programming

Code generation enables advantages, that are available to the generated code. By using meta programming more concepts can be expressed than the target language allows.

Code efficiency

A code generator has the privilege to build highly efficient code, by using the fastest statements, which have the fewest overhead. Modern programming language features like flexible syntax, polymorphism or reflection cause the application to run slower then needed[1]. A code generator can perform polymorphism and reflection on generation time and generate direct code, that is fast. The flexibility now lies in the model and not in the generated code, which tends to be impossible to maintain, because of the optimization potential. Code size can also be reduced, by only creating the needed parts.

Aspects of the applications functionality that normally would be scattered around in the code can be modeled centralized and then generated into the depended locations. An example would be a dependency framework, that is used by many classes. Instead of copy the import code to every class, it can be modeled as an decorator, that will provide the needed boilerplate code to all classes.

Platform language restrictions

Is is possible to break out of the target language's restrictions. The level of abstraction of the meta model and its defined language does not need to match that of the target language. A code generator can generate code that reflects higher level of features. It is possible to generate code that integrates the concept of OOP[2] into a non OOP-language. For example: A generator can generate structs and function pointers in a C/C++

1 according to [Stab] 181
2 Object Oriented Programming

application to imitate objects. The generated code on it's own is not considered safe or structured, but the static context analysis what is done while creating the model performs the sames checks as an OOP-compiler. Another example are generics, that can by used generating untyped objects[1] and perform a static context analysis while generating the code. Any feature that is known from modern languages can be used by imitating the compiler and its static context analysis. In a more extreme case, a virtual machine can be integrated into the application which can run any code.

Generated code with flat hierarchies is easy to analyze statically, because much of the logic is not abstracted into frameworks.

Compatibility

The higher the abstraction level of the meta model, the more platforms can be supported. The static context analysis is independent to the platform and should not integrate platform specific restrictions. A transition to another platform or supporting multiple platforms at once is possible and is one of the core strengths of MDD.

2.5.2 Generation

Now that a model exists, code can be generated out of it. Before code can be generated, the model has to be transformed into a generation oriented model. It's purpose is the simplification of the generation process by providing the target structure, which then can be converted to text directly. The transformed model is an model instance, that is targeting a specific platform. In cases of which the model's abstraction level is low and close to the target platforms, the transformation is not needed, since it brings a new level of complexity and abstraction to the process.

A typical example is a high level model, which could be implemented in any imperative language would be transformed into a UML class model. Object oriented programming code can then be generated out of the class model trivially. There are techniques to create program code from a transformed model. The two most popular ones[2] are templates- and API-based code generators.

1 *Object* (Java), *void∗* (C/C++), *any* (JavaScript)
2 according to [Stab] 186

Template Based Generators

A template is a definition, that is iterating over the model and prints the values on the corresponding locations in a text file. A template mixes the concrete target code presets with a markup notation, that is getting and writing values. The handlebar notation is one of the simplest form of template based code generation. A well formed model, which is considered the input is applied to a template, which results into a resulting output.

The following example tries to generate a HTML file with an overview of all used licenses of a mobile application, by using XSLT[1]. The model, which is created by a human user, contains only necessary information. Listing 2.1 shows an example, that consists of three dependencies, two of which use an MIT and the other an Apache license.

Listing 2.1: licences.xml - The model that is representing a licences web page

```
<licences>
  <carthage>
    <name>Alamofire</name>
    <url>https://github.com/Alamofire/Alamofire</url>
    <id>MIT</id>
    <copyright>Copyright (c) 2014-2018 Alamofire Software Foundation (
        http://alamofire.org/)</copyright>
  </carthage>
  <carthage>
    <name>Changeset</name>
    <url>https://github.com/osteslag/Changeset</url>
    <id>MIT</id>
    <copyright>Copyright (c) 2015-18 Joachim Bondo</copyright>
  </carthage>
  <carthage>
    <name>Couchbase Lite</name>
    <url>https://github.com/couchbase/couchbase-lite-ios</url>
    <id>Apache</id>
    <copyright>Version 2.0, January 2004 http://www.apache.org/licenses/<
        /copyright>
  </carthage>
</licences>
```

Now, a template and processor needs to create a HTML[2] file, that displays the dependencies and a copy of their respective license.

Listing 2.2: template.xslt - An XML file, that is later tranformed to XHTML

1 eXtensible Stylesheet Language Transformations
2 Hyper Text Markup Language

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
    Transform">
  <xsl:template match="/licences">
    <hr />
    <h1>iOS</h1>
    <hr />
    <xsl:apply-templates select="carthage"/>
  </xsl:template>
  <xsl:template match="carthage">
    <h3><xsl:value-of select="name" /></h3>
    <a>
    <xsl:attribute name="href"><xsl:value-of select="url" /></xsl:
        attribute>
    <xsl:value-of select="url" />
    </a>
    <h4><xsl:value-of select="id" /> License</h4>
    <p><xsl:value-of select="copyright" /></p>
    <xsl:if test="id = 'MIT'">
      <p>Permission is hereby granted ...</p>
    </xsl:if>
    <xsl:if test="id = 'Apache'">
      <p>TERMS AND CONDITIONS ...</p>
    </xsl:if>
    <hr />
  </xsl:template>
</xsl:stylesheet>
```

Listing 2.2 reveals some major advantages:

1. Changing the style and layout of the HTML file requires only a few changes. Adding or changing the CSS[1] classes of elements only requires to change the one occurrence in the template.

2. The URL[2] of an entry was displayed as the text content of an *a* element, but also occurs in the *href* attribute. Changing the URL later, can cause inconsistencies, if one of the two occurrences are omitted.

3. Instead of copying the license text to each entry, only the identifier (e.g. *MIT*) has to be defined. The template appends the correct text automatically.

4. Defining a template and a model of this size actually made the handling more easy and future changes can be made much faster, while keeping the file consistent.

1 Cascading Style Sheets
2 Unified Resource Locator

Since XSLT was created for web applications, PHP[1] offers native support and an XSLT processor. To generate the HTML file, the script in listing 2.3 can be ran.

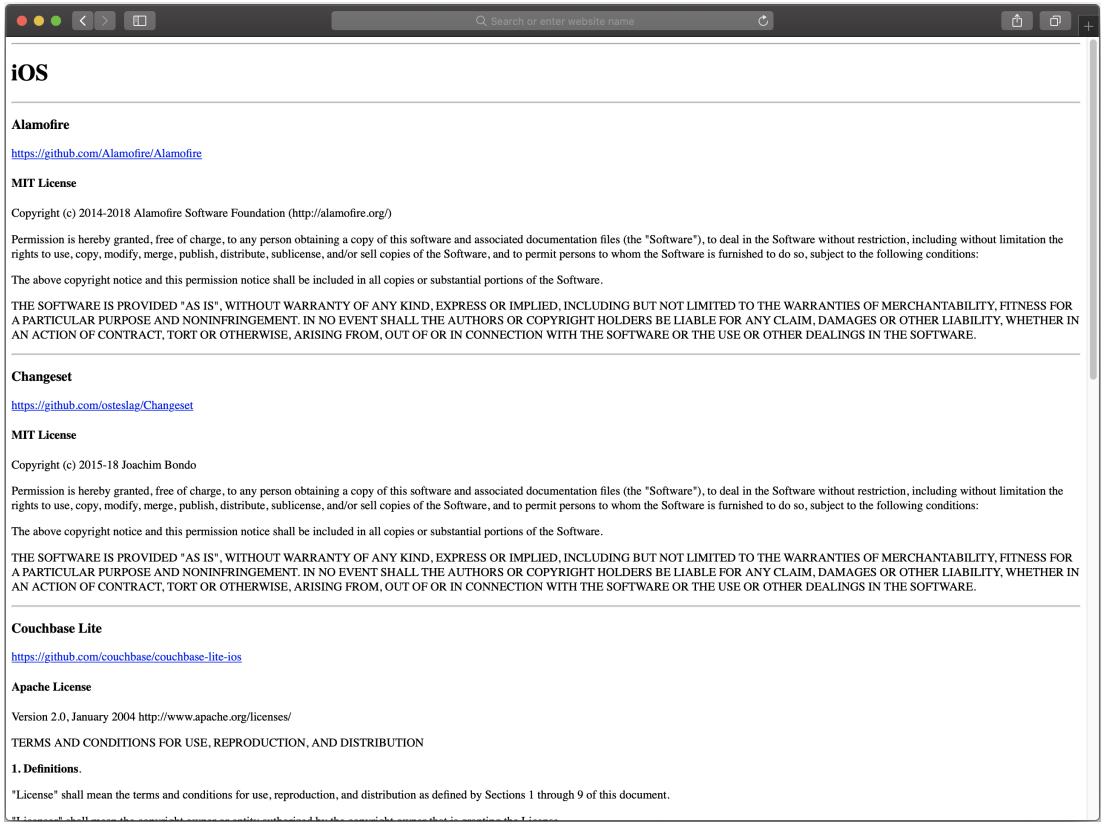
Listing 2.3: generate.php - Generation script

```
<?php
// Input
$xml = new DOMDocument;
$xml->load('licences.xml');

// Template
$xsl = new DOMDocument;
$xsl->load('template.xslt');

// Transformer
$proc = new XSLTProcessor;
$proc->importStyleSheet($xsl);
$output = $proc->transformToXML($xml);

// Write to file
$file = fopen("./LICENCES.html", "w") or die("Unable to open file!");
fwrite($file, $output);
fclose($file);
?>
```

Figure 2.3 displays the resulting HTML file. This file can now be bundled in to the app and enriched with CSS and be displayed in the apps license menu. The generation of the file can also be automated, by including the generation in the application's continues deployment pipeline. The model, the template and the generation script would be committed into version control. Another approach would bundle the template and the script into an external tool, which will be developed and maintained separately. In that case, only the model would be part of the version control and thus the application's source.

In larger projects, the template can become very complex. Using intermediate models as described in section 2.5.2 that are generated from the source model, can reduce the complexity, branching and nesting, because the intermediate model has either the same or a correspondingly pleasant structure. Nevertheless, templates will get very big very fast and are bound to the structure of the target platform. In [Stab] this approach is called "Templates and Metamodel", while *Metamodel* is the intermediate model, that is used to generate code directly.

1 PHP: Hypertext Preprocessor, earlier: Personal Home Page



iOS

Alamofire

https://github.com/Alamofire/Alamofire

MIT License

Copyright (c) 2014-2018 Alamofire Software Foundation (http://alamofire.org/)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Changeset

https://github.com/osteslag/Changeset

MIT License

Copyright (c) 2015-18 Joachim Bondo

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Couchbase Lite

https://github.com/couchbase/couchbase-lite-ios

Apache License

Version 2.0, January 2004 http://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

Figure 2.3: The generated HTML file viewed in a web browser

API-based Generators

The downside of templates are their untyped and unsafe character. It is possible to make small syntactic errors, that are unsafe and could cause the result to fail. Especially in interpreted target languages like PHP or HTML the error only occurs when the respective line is parsed or ran. Since the template elements are scattered in the file and important characters are hard to make out, this approach might be unsuitable for large projects.

An API-based generator is a programming library for a development language, which provides an interface to the AST[1]. It's purpose is to build up an AST using a type safe language like Java. Now the generator can be structured with Java's language features like Polymorphism, Interfaces and inheritance. The API makes sure, that only syntactically correct code is produced, because the AST does not allow it.

It is also possible to generate code for different target languages, which use the same AST. For example, it is possible to generate Kotlin instead of Java code, since Java is a

1 Abstract Syntax Tree

subset of Kotlin. Also, the .NET runtime makes it possible to generate Visual Basic, C# and C++ code from one AST.

Listing 2.4: Using an API to generate Java code, written in Swift

```
1 let studentClass = ClassDeclaration(named: "Student");
2 studentClass.baseType = personClass
3 studentClass.typeAttributes = .public
```

Listing 2.5: Java code result of lising 2.4

```
1 public class Student extends Person {
2 }
```

Listing 2.4 shows an example usage of an AST-based Java code generator, which is written in Java itself. This form of meta programming allows the creation of any valid code by using an object oriented interface. Since it is independent from any textual representation of the language, syntax errors in the generated code are impossible and any other language with an compatible AST can be generated.

The AST framework could make use of reflection to provide a more safe interface. For example, it could be possible to reference existing Java classes not by it's string name, but rather by using direct type references, like $typeof(System.Object)$. It might limit the set of compatible target languages, but in the example of Java, Kotlin is still compatible.

Using jQuery to manipulate or build HTML content is also a form of API-based code generators, since jQuery provides a safe[1] interface to the DOM[2]. The generated HTML code can then be read from the jQuery AST object.

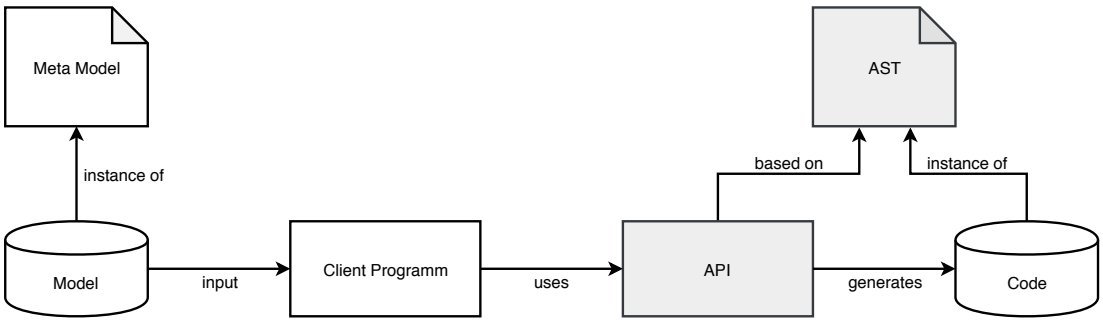


Figure 2.4: API based code generator infrastructure

1 Not all jQuery programming errors can be checked on compile time, despite a typed interface using TypeScript. Syntax errors in the generated code are not possible.
2 Document Object Model

Figure 2.4 illustrates the infrastructure of an API-based code generator. The darker rectangles (API and AST) represent components, that are independent to the domain, but are directly derived from the target platform. That means, that existing solutions[1] can be used.

This approach is suitable for large projects, that need a lot of generated code. If most of the generated code is boilerplate code, which has only few injected lines, templates might be the better approach. Since templates represent the resulting structure, they might be easy to read. API generators are more maintainable since changes are type safe and less sensitive to errors. The initial expense could be higher in comparison to templates. It is possible to mix these approaches by generating parts independently.

2.6 Model Interpreters

Instead of generating code, it is possible to run the model directly. The model is parsed by an interpreter, which uses the meta model to define an AST, which is the internal representation of a model. This representation can now be transformed be improve handling. It is possible to generate internal code out of the model which then is interpreted. Examples for that are Java or JavaScript, which are converted to byte code that is then ran. The actual interpreter of the byte code is independent to the original AST, so a more modular infrastructure is possible. Another approach is to run directly in the internal data structure. This methods seems easier at first, but ties the run time directly on the AST which makes maintaining and introducing changes harder.

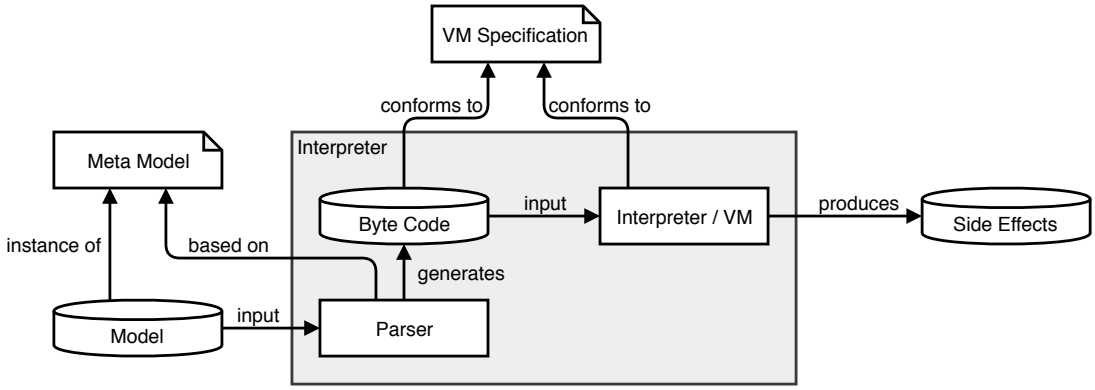


Figure 2.5: Interpreter infrastructure for common code interpreters

1 E. g. CodeDOM for .NET Applications `https://docs.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/using-the-codedom`

3 Concept

3.1 Implementation Goals

Same day delivery

easy configuration

simple datasources

common app tasks simplified

content driven (wordpress, social media)

reactive form elements

3.1.1 Requirements

3.1.2 Drawbacks

boundary conditions

limits to development of app contents (app is out, but the customer needs more functionality, which is not possible: incomprehensible reasons for limits.)

3.1.3 Iterative Development

4 Implementation

lorem ipsum

4.1 Language

4.2 Components

4.2.1 Sub-Models

4.2.2 Layout

4.2.3 Persistence

4.2.4 Interaction

4.3 Infrastructure

4.3.1 Control Center

4.3.2 Data Backend

4.3.3 Resource Server

4.3.4 Image Cruncher

5 Evaluation

lorem ispum

5.1 Requirements

5.2 Analysis

lorem ispum

5.2.1 Classification

5.2.2 Approach

5.2.3 Model

6 Conclusion

6.1 Summary

6.2 Discussion

6.3 Next Steps

6.3.1 Hybrid Approaches

6.3.2 Runtime Models

6.3.3 Abstraction

Bibliography

[Per] Perkins, Kleiner: Internet Trends Report 2018, URL https://www.kleinerperkins.com/perspectives/internet-trends-report-2018/

[Sei] Seidewitz, Ed: What Models Mean. URL https://www.researchgate.net/publication/3248044_What_models_mean

[Staa] Stachowiak, Herbert: *Allgemeine Modelltheorie*, URL http://archive.org/details/Stachowiak1973AllgemeineModelltheorie

[Stab] Stahl, Thomas und Völter, Markus: *Model-driven software development: technology, engineering, management*, John Wiley, OCLC: ocm64453466

Index of abbreviations

AST Abstract Syntax Tree

API Application Programming Interface

CSS Cascading Style Sheets

DSL Domain Specific Language

DOM Document Object Model

HTML Hyper Text Markup Language

MDA Model Driven Architecture

MDD Model Driven Development

MDSD Model Driven Software Development

mos Mobility Suite

OOP Object Oriented Programming

PHP PHP: Hypertext Preprocessor, earlier: Personal Home Page

UML Unified Modeling Language

URL Unified Resource Locator

XSLT eXtensible Stylesheet Language Transformations

List of Figures

1.1 Display notch with safe area, Apple iPhone X 2

2.1 Analyzing an existing implementation 10
2.2 Separation of code into source code and steady code 11
2.3 The generated HTML file viewed in a web browser 17
2.4 API based code generator infrastructure 18
2.5 Interpreter infrastructure for common code interpreters 19

List of Tables

Listings

2.1 licences.xml - The model that is representing a licences web page 14
2.2 template.xslt - An XML file, that is later tranformed to XHTML 14
2.3 generate.php - Generation script 16
2.4 Using an API to generate Java code, written in Swift 18
2.5 Java code result of lising 2.4 18

.